

FLASK-MONGODB PROJECT

This project is a full-stack web application developed using Flask and MongoDB Atlas. The application allows users to submit data through a frontend form, which is then stored securely in the MongoDB cloud database. Flask is used as the backend framework to handle routing, API requests, and database operations.

Features

- Flask backend with API routes*
- MongoDB Atlas cloud database integration*
- Frontend form for user data submission*
- Dynamic success and error handling*
- JSON response support*
- Clean project structure using VS*

MongoDB Atlas

Project Overview

DATABASE

Clusters

Search & Vector Search

Data Explorer

Backup

STREAMING DATA

Stream Processing

Triggers

SERVICES

AI Models

Migration

Data Federation

Visualization

SECURITY

Security Quickstart

Project Identity & Access

Database & Network Access

Activity Feed

ORGANIZATION

Shakti's Org - 2026-0...

PROJECT

Project 0

Clusters

Find a database deployment...

Edit Config

Create

Sample dataset successfully loaded. Get started with a sample query in [Data Explorer](#) or by connecting to the cluster.

Cluster1

Connect

View Monitoring

Browse Collections

Visualize Your Data

Build dashboards and charts, and embed them in your apps with MongoDB Charts.

Explore Charts

Dismiss

R 0

W

Last 46 minutes

0.05/s

Connections 0

Last 46 minutes

4.0

VERSION

REGION

TYPE

BACKUPS

LINKED APP SERVICES

8.0.23

AWS/Mumbai (ap-south-1)

Replica Set - 3 nodes

Inactive

None Linked

+ Add Tag

System Status: All Good

©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

EXPLORER

FLASK-APP

.venv

Include

Lib

Scripts

pyvenv.cfg

flask-mongo-atlas-demo

venv

app.py

data.json

app.py > ...

1 json

2 ask import Flask, render_template, request, jsonify

3 mongo import MongoClient

4

5 lask(__name__)

6

7 DB Atlas Connection

8 = MongoClient("mongodb+srv://Admin:Admin%40123@cluster1.jumillq.mongodb.net/?app

9

10 ient["studentdb"]

11 ion = db["students"]

12

13 Route

14 ute('/')

15 e():

16 urn render_template('index.html')

17

18 oute

19 ute('/api')

20 ():

21

22 :

23 with open('data.json', 'r') as file:

24 | data = json.load(file)

25

26 return jsonify(data)

27

28 ept Exception as e:

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

python

(venv) PS C:\Users\shakt\OneDrive\Desktop\Flask-app> python app.py

parser_shared.py", line 563, in _validate_uri

user, passwd = parse_userinfo(userinfo)

File "C:\Users\shakt\OneDrive\Desktop\Flask-app\venv\Lib\site-packages\pymongo\uri_parser_shared.py", line 136, in parse_userinfo

raise InvalidURI(

pymongo.errors.InvalidURI: Username and password must be escaped according to RFC 3986, use urllib.parse.quote_plus

(venv) PS C:\Users\shakt\OneDrive\Desktop\Flask-app> python app.py

* Serving Flask app 'app'

* Debug mode: on

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

* Running on http://127.0.0.1:5000

Press CTRL+C to quit

* Restarting with stat

* Debugger is active!

* Debugger PIN: 188-503-267

OUTLINE

TIMELINE

EXPLORER

FLASK-APP

.venv

> Include

> Lib

> Scripts

pyvenv.cfg

flask-mongo-atlas-demo

venv

app.py

data.json

Welcome

data.json

app.py

app.py > ...

```
1 import json
2 from flask import Flask, render_template, request, jsonify
3 from pymongo import MongoClient
4
5 app = Flask(__name__)
6
7 # MongoDB Atlas Connection
8 client = MongoClient("mongodb+srv://Admin:Admin%40123@cluster1.jumillq.mongodb.net/?appName=Cluster1")
9
10 db = client["studentdb"]
11 collection = db["students"]
12
13 # Home Route
14 @app.route('/')
15 def home():
16     return render_template('index.html')
17
18 # API Route
19 @app.route('/api')
20 def api():
21
22     try:
23         with open('data.json', 'r') as file:
24             data = json.load(file)
25
26         return jsonify(data)
27
28     except Exception as e:
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

python

(venv) PS C:\Users\shakt\OneDrive\Desktop\Flask-app> python app.py

File "C:\Users\shakt\OneDrive\Desktop\Flask-app\venv\Lib\site-packages\pymongo\synchronous\mongo_client.py", line 818, in __init__

res = _validate_uri(

File "C:\Users\shakt\OneDrive\Desktop\Flask-app\venv\Lib\site-packages\pymongo\uri_parser_shared.py", line 563, in _validate_uri

user, passwd = parse_userinfo(userinfo)

File "C:\Users\shakt\OneDrive\Desktop\Flask-app\venv\Lib\site-packages\pymongo\uri_parser_shared.py", line 136, in parse_userinfo

raise InvalidURI(

pymongo.errors.InvalidURI: Username and password must be escaped according to RFC 3986, use urllib.parse.quote_plus

(venv) PS C:\Users\shakt\OneDrive\Desktop\Flask-app> python app.py

* Serving Flask app 'app'

* Debug mode: on

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

* Running on http://127.0.0.1:5000

Press CTRL+C to quit

* Restarting with stat

* Debugger is active!

* Debugger PIN: 188-503-267

main*

Ln 8, Col 51

Spaces: 4

UTF-8

CRLF

{ }

Python

3.11.9 (venv)

.venv (3.11.9)

Go Live

Backend Development and MongoDB Integration

This section of the project demonstrates the backend implementation using Flask and the successful integration of MongoDB Atlas with the application. The backend handles API routing, database connectivity, and server-side operations through Python.

Key Functionalities

Flask application setup using Python
MongoDB Atlas cloud database connection
API route creation using Flask
JSON data handling and response generation
Backend debugging and error handling
Local server execution in development mode

Development Tools Used
Python Flask Framework
MongoDB Atlas
PyMongo Library
Visual Studio Code
Local Flask Development Server

Student Registration Form

Student Registration Form

Student Registration Form Interface

This module of the Flask-MongoDB project demonstrates the frontend user interface developed for student data registration. The form allows users to enter student details such as name, email, and course information through a simple and user-friendly interface.

Features

Student registration form

Input fields for user details

Submit button for data submission

Frontend and backend integration

Real-time data handling with Flask

MongoDB Atlas database storage

Flask Application Execution and Testing

This section shows the successful execution and testing of the Flask application inside Visual Studio Code. The terminal output confirms that the Flask development server is running properly on the local host environment.

Key Functionalities

Running Flask server in development mode

Handling HTTP GET and POST requests

Processing form submissions

Backend testing using localhost server

Database insertion using MongoDB Atlas

Debugging and server monitoring in VS Code

THANK YOU!

Feel free to approach us if you have any questions.